Kevin Küster

Zusammenfassung: Betriebssysteme

Betriebssysteme

Funktion

- Betriebssystem erweitert Hardware-Schnittstellen um abstrakte, anwendungsnähere Operationen, Beispiel: Festplattensektoren → Dateien [BS: Erweiterte Maschine]
- Betriebssystem verwaltet Betriebsmittel (Prozessor, Speicher, ...) für Programme
  - ↳ Zeitmultiplex: Verschiedene Programme erhalten Betriebsmittel zu verschiedenen Zeiten
  - ↳ Bereichsmultiplex: Programme erhalten Teilbereiche Gleichzeitig (Speicher, Bildschirm)

  [BS: Betriebsmittelverwalter]

BS-Arten

- Großrechner-BS: Stapelverarbeitung, Transaktionsbearbeitung, Timesharing
- Server-BS: Dienste für viele Clients (Unix, Linux, Win-Server)
- PC-BS: Einbenutzerbetrieb, Interaktivität (Win, MacOS)
- Realtime-BS: Zeitgarantien - hart (z.B. Flugzeug) vs. weich (z.B. Multimedia)
- Eingebettete Systeme: Ressourcen-Eingeschränkt (iOS, Android)
- SmartCard-BS: Sehr primitiv, chip-integriert
- Universal-BS: Unix-Familie + Windows

Arbeitsmodus

Arbeitsmodus ist Hardware-Mechanismus (Bit im PSW) → Kein BS-Mechanismus

- Systemmodus (privilegiert): Alle Aktionen erlaubt, voller Hardware-Zugriff
  - ↳ Nur BS (Kern) läuft
- Benutzermodus (eingeschränkt): Nur „normale" Befehle + Software außerhalb des Kerns

→ Privilegierter Befehl: Maschinenbefehl, den Hardware nur im Systemmodus ausführt

Übergang: Benutzer → System: Nur via Unterbrechung (Trap/Interrupt)

Übergang: System → Benutzer: Jederzeit möglich

Systemaufrufe

Auftrag von einem Programm an den BS-Kern

↳ Menge aller Systemaufrufe = BS-Schnittstelle (System-API)

Realisierung:

(i) Programm legt Parameter ab (Register/Stack)

(ii) Trap-Befehl löst Unterbrechung aus → Wechsel in Systemmodus

(iii) BS-Kern führt angeforderten Dienst aus

(iv) Rückkehr zum Programm in Benutzermodus

Systemaufruf selbst kein privilegierter Befehl

↳ Trap: Normaler Befehl; Bewirkt Systemmodus-Wechsel

Ebenen: Kommando (Benutzer) → Bibliotheksprozedur → eigentlicher Systemaufruf (Trap)

Unterbrechungen (Interrupts)

Unterbrechungen modifizieren die Programmausführung von außen:

- ↳ Externe Quellen: Geräte-Ereignisse, Fehler
- ↳ Prozessor selbst: Division durch 0, Trap-Befehl

Bei Unterbrechung: PC & PSW werden gesichert;

Arbeitsmodus → Systemmodus: Sprung in BS-Kern

Befehlzyklus: 1. Befehl holen → 2. Ausführung → 3. Unterbrechung prüfen → 4. Ggf. in BS-Kern wechseln

Prozesse & Threads

Prozess: Programm in Ausführung. Besitzt eigenen Adressraum, Betriebsmittel, Ausführungszustand. Zentrale Datenstruktur: Prozesskontrollblock (PCB) - enthält Prozess-ID (PID), Zustand, Register, Speicherinfo, offene Dateien, Prioritäten

Thread: Leichtgewichtiger Ausführungsstrang innerhalb eines Prozesses

↳ Threads eines Prozesses teilen: Adressraum, Programmcode, globale Variablen, offene Dateien, Nutzer-ID

↳ Threads haben eigene: Programmzähler, Kellerzeiger, Register, Stack

- Prozesserzeugung: fork() (Unix) - erzeugt Kinprozess als Kopie
  exec() - ersetzt Programm

Prozesszustände

- Laufend (running): Wird gerade vom Prozessor ausgeführt
- Bereit (ready): Lauffähig, wartet auf Prozessor-Zuteilung
- Blockiert (blocked/waiting): Wartet auf externes Ereignis (I/O)

Übergänge:

- Laufend → Bereit: Zeitscheibe abgelaufen (Scheduler entzieht Prozessor)
- Laufend → Blockiert: Prozessor wartet auf I/O oder Ereignis
- Blockiert → Bereit: Ereignis eingetreten (I/O) fertig
- Bereit → Laufend: Scheduler wählt Prozess aus
- Blockiert → Laufend: nicht Möglich (→ nur über Bereit)

Scheduling-Verfahren

Round Robin: Jeder Prozess erhält gleiche Zeitscheibe, dann kommt der nächste; Gleiche Aufteilung, aber nicht immer optimal

Priorität: Prozess mit höchster Priorität wird bevorzugt

↳ (Problem:) Verhungern (Starvation) möglich

⇒ FCFS (First Come First Served)

Interprozesskommunikation (IPC)

IPC: (i) Gemeinsamer Datenzugriff: Mehrere Prozesse nutzen gleiche Daten

(ii) Beschleunigung: Aufteilung einer Aufgabe auf parallele Prozesse

(iii) Modularität: Strukturierung in kooperierende Prozesse

Threats: (i), (ii), (iii); Gemeinsamer Datenzugriff → gleicher Adressraum

- Zeitkritischer Ablauf (Race Condition): Ergebnis hängt von zeitlicher Reihenfolge ab, in der Prozesse auf gemeinsame Daten zugreifen
- Kritischer Bereich: Codeabschnitt, in dem auf gemeinsame Daten zugegriffen wird
  - ↳ Gegenseitiger Ausschluss nötig

Synchronisationsmechanismen

TSL-Befehl (Test-and-Set-Lock): Atomarer Hardware-Befehl

- ↳ Liest ein Speicherwort und setzt es gleichzeitig auf '1'.
- ↳ Verhindert Race-Conditions auf Hardware-Ebene
- ↳ Nötig weil normales Lesen+Setzen durch Unterbrechungen getrennt werden kann

Semaphore (Dijkstra):

- Binäres Semaphor ('0'/'1') → Gegenseitiger Ausschluss (Mutex)
- Zählendes Semaphor → Beliebiger Wert; Ressourcenzählung
- down(s) → Wenn $s > 0$: Dekrementieren, Wenn $s = 0$: Blockieren
- up(s) → Wenn Prozesse warten: einen aufwecken, sonst: Inkrementieren
  - ↳ down(s) & up(s) immer Atomar

Monitor: Datenstruktur mit Prozeduren, von denen zu jeder Zeit nur eine aktiv sein kann. Automatischer gegenseitiger Ausschluss; In Java: Synchronized

Interrupt-Sperre: Unterbrechungen abschalten → kein Prozesswechsel möglich → kritischer Bereich geschützt ⇒ Nur auf Einprozessorsystemen wirksam & nur im Systemmodus verwendbar

Klassische Probleme

Erzeuger-Verbraucher-Problem: Erzeuger produziert in Puffer, Verbraucher entnimmt; Synchronisation nötig bei vollem/leerem Puffer:

Lösung: Semaphore (full, empty, mutex)

Virtuelle Adressierung & Seitenadressierung

Virtuelle Adressen:
↳ Programme verwenden „vorläufige" Adressen (ab 0).
↳ Memory Management Unit (MMU) übersetzt in physikalische Adressen

Seitenadressierung (Paging):
↳ Virtueller Adressraum in Seiten (Pages) gleicher Größe unterteilt
↳ Physischer Speicher in Seitenrahmen (Frames) gleicher Größe
↳ Seitentabelle: Abbildung Seitennummer → Seitenrahmennummer

Adressraumrechnung:
(i) Virtuelle Adresse aufteilen: Obere Bits: Seitennummer, untere Bits: Offset
(ii) Seitennummer → Seitentabelle → Seitenrahmennummer
(iii) Physikalische Adresse = Seitenrahmennummer · Seitengröße + Offset

Beispiel

Seitengröße: 4 KiB; Offset = untere 12 Bits => 0x0FFF = 4095

Basisadressierung & Relokation

Basisadressierung: Einfachste MMU-Variante
↳ Basisregister enthält Startadresse des Programms in Speicher
↳ Limitregister enthält Endadresse (Schutz)
↳ Physische Adresse = Virtuelle Adresse + Basisregister
↳ Schutz: Prüfung ob Ergebnis < Limitregister

Relokation: Das gleiche Programm wird an verschiedenen Stellen im Hauptspeicher ausgeführt => Erfordert virtuelle Adressierung

Seitenersetzung & TLB

Wenn alle Seitenrahmen belegt sind und neue Seite geladen werden muss: Eine Seite muss verdrängt werden

Lokale Seitenersetzung: Nur Seiten des gleichen Prozesses werden verdrängt.
↳ Gefahr: Prozess mit zu wenig Rahmen: Ständige Seitenfehler (Trashing)

Algorithmen: FIFO, LRU (Least Recently Used), Second Chance, optimal

TLB (Translation Lookaside Buffer): Cache für Seitentabellen-Einträge
↳ Beschleunigt Adressraumrechnung

Geräteverwaltung

Controller-Register-Adressierung

(i) Eigener I/O-Adressraum: Spezielle I/O-Befehle (IN/OUT) adressieren Controller-Register in seperatem Adressraum

(ii) Memory-Mapped I/O: Controller-Register sind in den Hauptspeicher-Adressraum eingeblendet => Zugriff über normale Speicherbefehle (MOV)

Gerätezugriffsarten

(i) Direkter Zugriff (Programmierte I/O Polling):
↳ Prozessor überträgt Daten selbst; wartet aktiv auf Bereitschaft (Busy Waiting)
↳ pro: Einfach, kein zusätzlicher Hardware-Aufwand
↳ contra: Prozessor blockiert (Busy Waiting); keine Parallelarbeit
↳ Sinnvoll bei: Schnellen Geräten mit geringer Datenmenge

(ii) Verwendung von Unterbrechungen (Interrupt-gesteuert):
↳ Prozessor startet Auftrag, macht anderes, Controller meldet Fertigstellung via Interrupt
↳ pro: Parallelität → kein Busy Waiting
↳ contra: Overhead für Interrupt-Behandlung bei jedem Datenwort
↳ Sinnvoll bei: Geräte mit langer Verarbeitungszeit

(iii) DMA (Direct Memory Access):
↳ DMA Controller übernimmt kompletten Datentransport zwischen Geräten und Speicher
↳ Prozessor nur am Anfang (Auftrag) und Ende (Interrupt) beteiligt
↳ pro: Prozessor fast komplett frei, effizient für große Datenmengen
↳ contra: Zusätzliche Hardware (DMA-Controller)

Dateisysteme

Dateiblock-Verwaltung

Problem: Wie werden die zu einer Datei gehörigen Blöcke auf der Platte verwaltet?

(i) Kontinuierliche Allokation: Datei liegt in zusammenhängenden Blöcken
↳ pro: Einfacher, schneller sequentieller Zugriff
↳ contra: Fragmentierung, Größe muss vorab bekannt sein
↳ Einsatz: CD-ROM, Medien-Dateien

(ii) Verkettete Liste: Jeder Block enthält Zeiger auf nächsten Block
↳ pro: Kein externes Fragmentieren
↳ contra: Langsamer wahlfreier Zugriff, Zeiger belegt Platz im Block

(iii) FAT (File Allocation Table): Verkettung in seperater Tabellen
↳ pro: Schnellerer Zugriff als reine Verkettung; Block komplett für Daten nutzbar
↳ contra: FAT muss komplett im RAM liegen (=> groß)

(iv) Indexknoten (Inodes): Jede Datei hat eigenen Indexblock mit Blocknummern
↳ pro: Nur Inode der geöffneten Dateien im RAM (nicht ganze FAT) → Schnellerer Direktzugriff
↳ contra: Zusätzlicher Speicher für Inodes. Bei sehr großen Dateien: Indirekte Blöcke nötig

FAT-Berechnung

(i) Anzahl Blöcke = Partitionsgröße / Blockgröße
(ii) Bits pro FAT-Eintrag ≥ $\log_2$(Anzahl Blöcke) → aufrunden nächste Zweierpotenz
(iii) FAT-Größe: Anzahl Blöcke · Bits pro Einheit

Beispiel

16 GB Partition, 1 KB Blöcke:
(i) Anzahl Blöcke = $\frac{16\ GiB}{1\ KiB} = \frac{2^{34}}{2^{10}} = 2^{24} = 16\ M$ Blöcke
(ii) Bits pro Eintrag ≥ 24 → min. 24 Bit (3 Byte)
(iii) FAT-Größe = 16 M · 3 Byte = 48 MiB

Journaling

Problem: Dateisystem-Änderungen erfordern mehrere Schritte
↳ Absturz mittendrin => Inkonsistenz

Lösung: Journaling
↳ Vor Ausführung: Beschreibung der Änderung ins Journal schreiben
↳ Ausführung der Schritte
↳ Nach Abschluss: Journaleintrag löschen

Bei Absturz: Journal-Einträge wiederholen → Konsistenz wiederherstellen:
↳ Schritte müssen Idempotent sein!

- Vollständiges Journaling: Alle Blöcke ins Journal (sicher, aber langsam)
- Metadaten-Journaling: Nur Verwaltungsdaten ins Journal (schneller, Dateiinhalte können verloren gehen)